TASK – 1

Write a program to demonstrate different number data types in Python.

```
integer_num = 10          # Integer data type
float_num = 10.5          # Float data type
complex_num = 3 + 4j      # Complex data type
boolean_num = True        # Boolean data type

# Display the values and their data types
print("Integer:", integer_num)
print("Type:", type(integer_num))

print("\nFloat:", float_num)
print("Type:", type(float_num))

print("\nComplex:", complex_num)
print("Type:", type(complex_num))

print("\nBoolean:", boolean_num)
print("Type:", type(boolean_num))
```

OUTPUT

```
Integer: 10
Type: <class 'int'>

Float: 10.5
Type: <class 'float'>

Complex: (3+4j)
Type: <class 'complex'>

Boolean: True
Type: <class 'bool'>
```

Figure 1: Output of task 1

TASK – 2

Write a program to perform different Arithmetic Operations on numbers in Python.

```
# Taking input from the user
num1 = int(input("Enter the first number: "))
num2 = int(input("Enter the second number: "))

# Performing arithmetic operations
print("Addition:", num1 + num2)
print("Subtraction:", num1 - num2)
print("Multiplication:", num1 * num2)

# Checking to avoid division by zero
if num2 != 0:
    print("Division:", num1 / num2)
    print("Floor Division:", num1 // num2)
    print("Modulus:", num1 % num2)
else:
    print("Division: Cannot divide by zero")
    print("Floor Division: Cannot divide by zero")
    print("Modulus: Cannot divide by zero")

# Exponentiation
print("Exponentiation:", num1 ** num2)
```

OUTPUT

```
Enter the first number: 36
Enter the second number: 6
Addition: 42
Subtraction: 30
Multiplication: 216
Division: 6.0
Floor Division: 6
Modulus: 0
Exponentiation: 2176782336
```

Figure 2: Output of task 2

TASK – 3

Write a program to create, concatenate and print a string and accessing sub-string from a given string.

```
# Creating two strings
str1 = input("Enter the first string: ")
str2 = input("Enter the second string: ")

result = str1 + " " + str2              # Concatenating the strings
print("Concatenated string:", result)

# Accessing a substring using slicing
start = int(input("Enter the starting index of substring: "))
end = int(input("Enter the ending index of substring: "))

substring = result[start:end]
print("Substring:", substring)
```

OUTPUT

```
Enter the first string: DAVIET
Enter the second string: Jalandhar
Concatenated string: DAVIET Jalandhar
Enter the starting index of substring: 3
Enter the ending index of substring: 11
Substring: IET Jala
```

Figure 3: Output of task 3

TASK – 4

Write a python script to print the current date in the following format "Sun May 29 02:26:23 IST 2017"

```
import datetime

now = datetime.datetime.now()       # Get the current date and time

# Format and print the current date and time
print("Expected Output: ")
print(now.strftime("%a %b %d %H:%M:%S IST %Y"))
```

OUTPUT

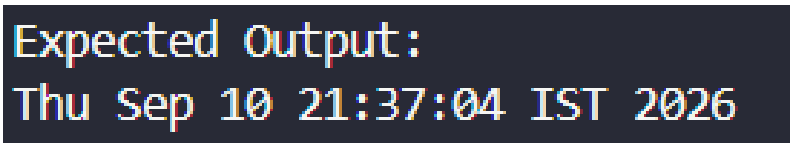


Figure 4: Output of task 4

TASK – 5

Write a program to create, append, and remove lists in python.

```
# Taking list elements as input from the user
my_list = list(
    map(
        int,
        input("Enter list elements separated by space: ").split()
    )
)
print("Original list:", my_list)

# Appending an element to the list
element = int(input("Enter an element to append: "))
my_list.append(element)
print("List after appending:", my_list)

# Removing an element from the list
element = int(input("Enter an element to remove: "))

if element in my_list:
    my_list.remove(element)
    print("List after removing:", my_list)
else:
    print("Element not found in the list")
```

OUTPUT

```
Enter list elements separated by space: 36 78 67 93 28 31
Original list: [36, 78, 67, 93, 28, 31]
Enter an element to append: 6
List after appending: [36, 78, 67, 93, 28, 31, 6]
Enter an element to remove: 93
List after removing: [36, 78, 67, 28, 31, 6]
```

Figure 5: Output of task 5

TASK – 6

Write a program to demonstrate working with tuples in python.

```
# Taking tuple elements as input from the user
my_tuple = tuple(map(int, input("Enter tuple elements separated by space: ").split()))

print("Original tuple:", my_tuple)

# Accessing tuple elements
print("First element:", my_tuple[0])
print("Last element:", my_tuple[-1])

# Slicing the tuple
print("Sliced tuple:", my_tuple[1:])

# Length of tuple
print("Length of tuple:", len(my_tuple))

# Counting occurrences of an element
element = int(input("Enter an element to count: "))
print("Number of occurrences:", my_tuple.count(element))

# Finding the index of an element
element = int(input("Enter an element to find its index: "))

if element in my_tuple:
    print("Index of element:", my_tuple.index(element))
else:
    print("Element not found in tuple")

# Repeating a tuple
n = int(input("Enter number of times to repeat the tuple: "))
print("Repeated tuple:", my_tuple * n)

# Finding maximum and minimum values
print("Maximum element:", max(my_tuple))
print("Minimum element:", min(my_tuple))

# Finding sum of tuple elements
print("Sum of elements:", sum(my_tuple))

# Converting tuple to list
tuple_list = list(my_tuple)
print("Tuple converted to list:", tuple_list)

# Concatenating two tuples
second_tuple = tuple(map(int, input("Enter elements for another tuple: ").split()))
combined_tuple = my_tuple + second_tuple
print("Concatenated tuple:", combined_tuple)
```

OUTPUT

```
Enter tuple elements separated by space: 78 93 46 29 51 73 73 46
Original tuple: (78, 93, 46, 29, 51, 73, 73, 46)
First element: 78
Last element: 46
Sliced tuple: (93, 46, 29, 51, 73, 73, 46)
Length of tuple: 8
Enter an element to count: 46
Number of occurrences: 2
Enter an element to find its index: 51
Index of element: 4
Enter number of times to repeat the tuple: 2
Repeated tuple: (78, 93, 46, 29, 51, 73, 73, 46, 78, 93, 46, 29, 51, 73, 73, 46)
Maximum element: 93
Minimum element: 29
Sum of elements: 489
Tuple converted to list: [78, 93, 46, 29, 51, 73, 73, 46]
Enter elements for another tuple: 67 52 19 6 36
Concatenated tuple: (78, 93, 46, 29, 51, 73, 73, 46, 67, 52, 19, 6, 36)
```

Figure 6: Output of task 6

TASK – 7

Write a program to demonstrate working with dictionaries in python.

```
n = int(input("Enter the number of key-value pairs: "))

my_dict = {}

for i in range(n):
    key = input("Enter key: ")
    value = input("Enter value: ")
    my_dict[key] = value

print("\nOriginal dictionary:", my_dict)

# Accessing a value using a key
key = input("\nEnter a key to access its value: ")
if key in my_dict:
    print("Value:", my_dict[key])
else:
    print("Key not found")

# Adding a new key-value pair
key = input("\nEnter a new key: ")
value = input("Enter its value: ")
my_dict[key] = value
print("Dictionary after adding:", my_dict)

# Updating a value
key = input("\nEnter a key to update: ")
if key in my_dict:
    value = input("Enter the new value: ")
    my_dict.update({key: value})
    print("Dictionary after updating:", my_dict)
else:
    print("Key not found")

# Using get() method
key = input("\nEnter a key to use get(): ")
print("Value:", my_dict.get(key, "Key not found"))

# Displaying all keys
print("\nKeys:", my_dict.keys())

# Displaying all values
print("Values:", my_dict.values())

# Displaying all key-value pairs
print("Items:", my_dict.items())
```

```
# Checking whether a key exists
key = input("\nEnter a key to search: ")
if key in my_dict:
    print("Key exists in the dictionary")
else:
    print("Key does not exist in the dictionary")

# Removing an element using pop()
key = input("\nEnter a key to remove using pop(): ")
if key in my_dict:
    removed_value = my_dict.pop(key)
    print("Removed value:", removed_value)
    print("Dictionary after pop():", my_dict)
else:
    print("Key not found")

# Removing the last key-value pair using popitem()
if my_dict:
    removed_item = my_dict.popitem()
    print("\nRemoved last item using popitem():", removed_item)
    print("Dictionary after popitem():", my_dict)
else:
    print("\nDictionary is empty")

# Creating a copy of the dictionary
dict_copy = my_dict.copy()
print("\nCopy of dictionary:", dict_copy)

# Creating a dictionary using fromkeys()
keys = input("\nEnter keys for fromkeys() separated by space: ").split()
default_value = input("Enter the common value: ")
new_dict = dict.fromkeys(keys, default_value)
print("Dictionary using fromkeys():", new_dict)

# Clearing the dictionary
my_dict.clear()
print("\nDictionary after clear():", my_dict)
```

OUTPUT

```
Enter the number of key-value pairs: 4
Enter key: Name
Enter value: Abhi Saurav Saroya
Enter key: Class
Enter value: BTech CSE AI-ML
Enter key: Semester
Enter value: 5-A
Enter key: Group
Enter value: A

Original dictionary: {'Name': 'Abhi Saurav Saroya', 'Class': 'BTech CSE AI-ML', 'Semester': '5-A', 'Group': 'A'}
```

```
Enter a key to access its value: Name
Value: Abhi Saurav Saroya

Enter a new key: CGPA
Enter its value: 8.0+
Dictionary after adding: {'Name': 'Abhi Saurav Saroya', 'Class': 'BTech CSE AI-ML', 'Semester': '5-A', 'Group': 'A', 'CGPA': '8.0+'}

Enter a key to update: Class
Enter the new value: BTech CSE (AI & ML)
Dictionary after updating: {'Name': 'Abhi Saurav Saroya', 'Class': 'BTech CSE (AI & ML)', 'Semester': '5-A', 'Group': 'A', 'CGPA': '8.0+'}

Enter a key to use get(): Semester
Value: 5-A
Keys: dict_keys(['Name', 'Class', 'Semester', 'Group', 'CGPA'])
Values: dict_values(['Abhi Saurav Saroya', 'BTech CSE (AI & ML)', '5-A', 'A', '8.0+'])
Items: dict_items([('Name', 'Abhi Saurav Saroya'), ('Class', 'BTech CSE (AI & ML)'), ('Semester', '5-A'), ('Group', 'A'), ('CGPA', '8.0+')])

Enter a key to search: Marks
Key does not exist in the dictionary

Enter a key to remove using pop(): Group
Removed value: A
Dictionary after pop(): {'Name': 'Abhi Saurav Saroya', 'Class': 'BTech CSE (AI & ML)', 'Semester': '5-A', 'CGPA': '8.0+'}

Removed last item using popitem(): ('CGPA', '8.0+')
Dictionary after popitem(): {'Name': 'Abhi Saurav Saroya', 'Class': 'BTech CSE (AI & ML)', 'Semester': '5-A'}

Copy of dictionary: {'Name': 'Abhi Saurav Saroya', 'Class': 'BTech CSE (AI & ML)', 'Semester': '5-A'}
Enter keys for fromkeys() separated by space: City Location
Enter the common value: Jalandhar
Dictionary using fromkeys(): {'City': 'Jalandhar', 'Location': 'Jalandhar'}

Dictionary after clear(): {}
```

Figure 7: Output of task 7

TASK – 8

Write a python program to find largest of three numbers.

```
num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))
num3 = float(input("Enter the third number: "))

# Method 1: Using if-elif-else
if num1 >= num2 and num1 >= num3:
    largest = num1
elif num2 >= num1 and num2 >= num3:
    largest = num2
else:
    largest = num3

print("Largest number using if-else:", largest)

# Method 2: Using max() function
largest = max(num1, num2, num3)
print("Largest number using max():", largest)
```

OUTPUT

```
Enter the first number: 36.6
Enter the second number: 98.63
Enter the third number: 17.82
Largest number using if-else: 98.63
Largest number using max(): 98.63
```

Figure 8: Output of task 8

TASK – 9

Write a Python program to convert temperatures to and from Celsius, Fahrenheit.

```
def celsius_to_fahrenheit(c: float) -> float:
    """
       Function to convert Celsius to Fahrenheit
    """
    return (c * 9 / 5) + 32

def fahrenheit_to_celsius(f: float) -> float:
    """
       Function to convert Fahrenheit to Celsius
    """
    return (f - 32) * 5 / 9

# Taking input from the user
choice = input("Enter C to convert Celsius to Fahrenheit or F to convert Fahrenheit to Celsius:
")

if choice.strip().upper() == "C":
    celsius = float(input("Enter temperature in Celsius: "))
    fahrenheit = celsius_to_fahrenheit(celsius)
    print("Temperature in Fahrenheit:", fahrenheit)

elif choice.strip().upper() == "F":
    fahrenheit = float(input("Enter temperature in Fahrenheit: "))
    celsius = fahrenheit_to_celsius(fahrenheit)
    print("Temperature in Celsius:", celsius)

else:
    print("Invalid choice")
```

OUTPUT

```
Enter C to convert Celsius to Fahrenheit or F to convert Fahrenheit to Celsius: f
Enter temperature in Fahrenheit: 102.35
Temperature in Celsius: 39.083333333333336
```

```
Enter C to convert Celsius to Fahrenheit or F to convert Fahrenheit to Celsius: c
Enter temperature in Celsius: 42.98
Temperature in Fahrenheit: 109.364
```

Figure 9: Output of task 9

TASK – 10

Write a Python program to construct the left aligned full pyramid pattern, using a nested for loop.

```
# Increasing number of stars
for i in range(1, 5):
    for j in range(i):
        print("*", end=" ")
    print()

# Decreasing number of stars
for i in range(3, 0, -1):
    for j in range(i):
        print("*", end=" ")
    print()
```

OUTPUT



Figure 10: Output of task 10

TASK – 11

Write a Python script that prints prime numbers less than 20.

```
# Function to check whether a number is prime
def is_prime(num: int) -> bool:
    for i in range(2, num):
        if num % i == 0:
            return False

    return True

# Printing prime numbers less than 20
print("Prime numbers less than 20:")

for num in range(2, 20):
    if is_prime(num):
        print(num, end=" ")
```

OUTPUT

```
Prime numbers less than 20:
2 3 5 7 11 13 17 19
```

Figure 11: Output of task 11

TASK – 12

Write a python program to find factorial of a number using Recursion.

```
def factorial(n: int) -> int:
    '''
        Function to calculate factorial of an integer.
    '''
    if n == 0 or n == 1:
        return 1
    else:
        return n * factorial(n - 1)

num = int(input("Enter a number: "))

if num < 0:
    print("Factorial is not defined for negative numbers")
else:
    print("Factorial of", num, "is:", factorial(num))
```

OUTPUT

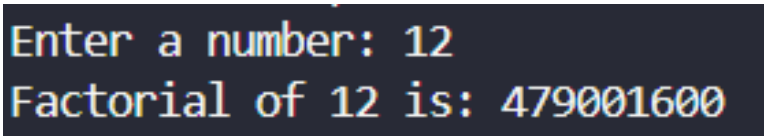


Figure 12: Output of task 12

TASK – 13

Write a program that accepts the lengths of three sides of a triangle as inputs. The program output should indicate whether or not the triangle is a right triangle.

```
a = float(input("Enter the length of first side: "))
b = float(input("Enter the length of second side: "))
c = float(input("Enter the length of third side: "))

# Finding the largest side
largest = max(a, b, c)

# Checking the Pythagorean theorem
if largest == a:
    other1, other2 = b, c
elif largest == b:
    other1, other2 = a, c
else:
    other1, other2 = a, b

if a + b > c and a + c > b and b + c > a and largest ** 2 == other1 ** 2 + other2 ** 2:
    print("The triangle is a right triangle.")
else:
    print("The triangle is not a right triangle.")
```

OUTPUT

```
Enter the length of first side: 5
Enter the length of second side: 3
Enter the length of third side: 4
The triangle is a right triangle.
```

Figure 13: Output of task 13

TASK – 14

Write a python program to define a module to find Fibonacci Numbers and import the module to another program.

fibonacci.py

```
def fibonacci(n):
    a = 0
    b = 1

    for i in range(n):
        print(a, end=" ")
        a, b = b, a + b
```

main.py

```
import fibonacci as fibo

# Taking input from the user
n = int(input("Enter the number of terms: "))

# Calling the function from the module
print("Fibonacci series:")
fibo.fibonacci(n)
```

OUTPUT

```
Enter the number of terms: 15
Fibonacci series:
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

Figure 14: Output of task 14

TASK – 15

Write a python program to define a module and import a specific function in that module to another program.

calculator.py

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    return a / b
```

main.py

```
# Importing a specific function from the module
from calculator import multiply

num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))

# Calling the imported function
result = multiply(num1, num2)
print("Product:", result)
```

OUTPUT

```
Enter the first number: 76.5
Enter the second number: 34.25
Product: 2620.125
```

Figure 15: Output of task 15

TASK – 16

Write a script named copyfile.py. This script should prompt the user for the names of two text files. The contents of the first file should be input and written to the second file.

```
import os
folder = os.path.dirname(os.path.abspath(__file__))

# Taking file names from the user
source_file = input("Enter the name of the source file: ")
destination_file = input("Enter the name of the destination file: ")

# Creating the complete file paths
source_path = os.path.join(folder, source_file)
destination_path = os.path.join(folder, destination_file)

# Opening the source file in read mode
with open(source_path, "r") as file1:
    content = file1.read()

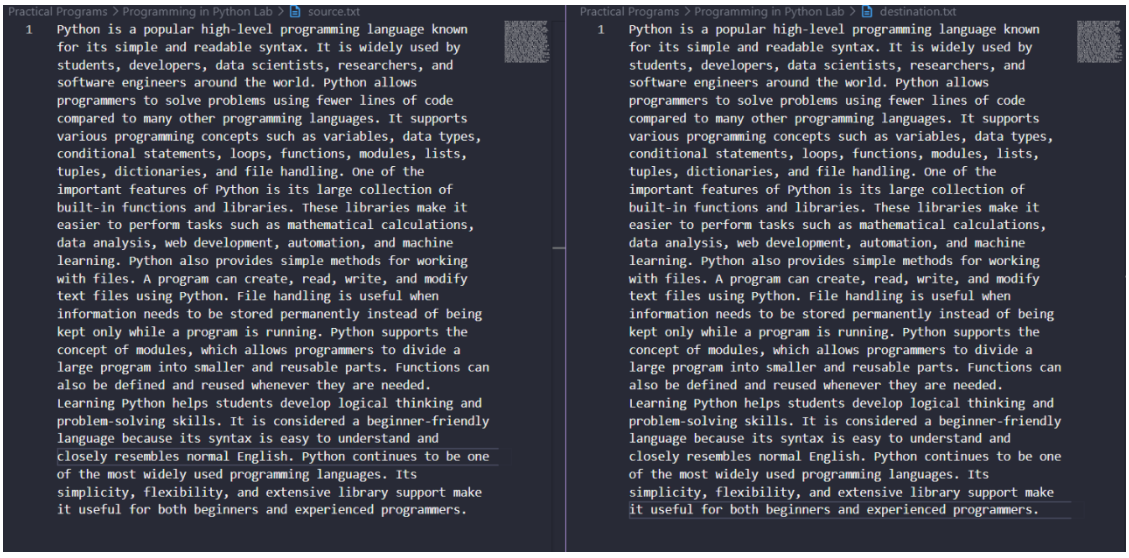
# Opening the destination file in write mode
with open(destination_path, "w") as file2:
    file2.write(content)

print("Contents copied successfully.")
```

OUTPUT

```
Enter the name of the source file: source.txt
Enter the name of the destination file: destination.txt
Contents copied successfully.
```

Figure 16: Output of task 16



TASK – 17

Write a program that inputs a text file. The program should print all of the unique words in the file in alphabetical order.

```
import os
import string

folder = os.path.dirname(os.path.abspath(__file__))

file_name = input("Enter the name of the text file: ")
file_path = os.path.join(folder, file_name)

with open(file_path, "r") as file:
    text = file.read()

# Remove punctuation from the text
text = text.translate(str.maketrans("", "", string.punctuation))

words = text.split()
unique_words = set(word.lower() for word in words)
sorted_words = sorted(unique_words)

print("Unique words in alphabetical order:")
for word in sorted_words:
    print(word, end=", ")
```

OUTPUT

```
Enter the name of the text file: source.txt
Unique words in alphabetical order:
a, allows, also, analysis, and, are, around, as, automation, be, because, beginnerfriendly, beginners, being, both, builtin, by, calculations, can, closely, code,
collection, compared, concept, concepts, conditional, considered, continues, create, data, defined, develop, developers, development, dictionaries, divide, easier,
 easy, engineers, english, experienced, extensive, features, fewer, file, files, flexibility, for, functions, handling, helps, highlevel, important, information, i
nstead, into, is, it, its, kept, known, language, languages, large, learning, libraries, library, lines, lists, logical, loops, machine, make, many, mathematical,
methods, modify, modules, most, needed, needs, normal, of, one, only, other, parts, perform, permanently, popular, problems, problemsolving, program, programmers,
programming, provides, python, read, readable, researchers, resembles, reusable, reused, running, scientists, simple, simplicity, skills, smaller, software, solve,
 statements, stored, students, such, support, supports, syntax, tasks, text, the, these, they, thinking, to, tuples, types, understand, used, useful, using, variab
les, various, web, when, whenever, which, while, widely, with, working, world, write,
```

Figure 17: Output of task 17

TASK – 18

Write a Python class to convert an integer to a roman numeral.

```
class RomanNumeral:
    I = 1
    V = 5
    X = 10
    L = 50
    C = 100
    D = 500
    M = 1000

    def __init__(self, number):
        self.number = number

    def convert(self) -> str:
        number = self.number
        roman = ""

        # Thousands
        while number >= self.M:
            roman += "M"
            number -= self.M

        # Hundreds
        if number >= 900:
            roman += "CM"
            number -= 900
        elif number >= self.D:
            roman += "D"
            number -= self.D
        elif number >= 400:
            roman += "CD"
            number -= 400

        while number >= self.C:
            roman += "C"
            number -= self.C

        # Tens
        if number >= 90:
            roman += "XC"
            number -= 90
        elif number >= self.L:
            roman += "L"
            number -= self.L
        elif number >= 40:
            roman += "XL"
            number -= 40
```

```
        while number >= self.X:
            roman += "X"
            number -= self.X

        # Units
        if number == 9:
            roman += "IX"
            number -= 9
        elif number >= self.V:
            roman += "V"
            number -= self.V
        elif number == 4:
            roman += "IV"
            number -= 4

        while number >= self.I:
            roman += "I"
            number -= self.I

        return roman

number = int(input("Enter an integer: "))

if 1 <= number <= 3999:
    obj = RomanNumeral(number)
    print("Roman numeral:", obj.convert())
else:
    print("Please enter a number between 1 and 3999.")
```

OUTPUT

```
Enter an integer: 89
Roman numeral: LXXXIX
```

```
Enter an integer: 793
Roman numeral: DCCXCIII
```

Figure 18: Output of task 18

TASK – 19

Write a Python class to implement pow(x, n)

```
class Power:
    # Constructor to initialize x and n
    def __init__(self, x: int, n: int):
        self.x = x
        self.n = n

    def pow(self) -> int:
        return self.x ** self.n

x = int(input("Enter the value of x: "))
n = int(input("Enter the value of n: "))

# Creating an object of Power class
p = Power(x, n)
print(f"pow({x}, {n}) =", p.pow())
```

OUTPUT

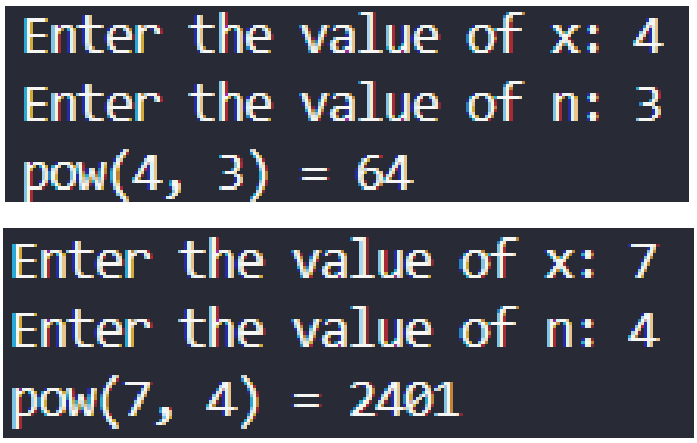


Figure 19: Output of task 19

TASK – 20

Write a Python class to reverse a string word by word.

```
class ReverseString:
    # Constructor to initialize the string
    def __init__(self, text):
        self.text = text

    # Function to reverse the string word by word
    def reverse(self):
        words = self.text.split()
        words.reverse()
        return " ".join(words)

text = input("Enter a string: ")

obj = ReverseString(text)
print("Reversed string:", obj.reverse())
```

OUTPUT

```
Enter a string: This is task 20 of PIP Lab
Reversed string: Lab PIP of 20 task is This
```

Figure 20: Output of task 20

TASK – 21

To read and analyze a CSV or Excel file containing two columns using Pandas and visualize the data using data visualization libraries.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

```
df = pd.read_csv("steps_calories.csv")
```

```
print("First five rows of the dataset:")
print(df.head())

print("\nDataset information:")
print(df.info())

print("\nStatistical summary:")
print(df.describe())

print("\nNumber of rows and columns:")
print(df.shape)
```

```
First five rows of the dataset:
   Steps  Calories_Burned
0   8270              328
1   1860               92
2   6390              282
3  14418              550
4   6191              265

Dataset information:
<class 'pandas.DataFrame'>
RangeIndex: 400 entries, 0 to 399
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Steps            400 non-null    int64
 1   Calories_Burned  400 non-null    int64
dtypes: int64(2)
memory usage: 6.4 KB
None

Statistical summary:
             Steps  Calories_Burned
count    400.000000       400.000000
mean    8099.972500       324.805000
std     3973.146419       161.836723
min     1064.000000        20.000000
25%     4735.000000       193.750000
50%     7992.000000       320.000000
75%    11639.500000       456.000000
max    14992.000000       786.000000

Number of rows and columns:
(400, 2)
```

```
print("Missing values:")
print(df.isnull().sum())

print("\nDuplicate rows:", df.duplicated().sum())

df = df.dropna(subset=["Steps", "Calories_Burned"])
df = df.drop_duplicates()

print("\nDataset after cleaning:")
print(df.shape)
```

```
Missing values:
Steps              0
Calories_Burned    0
dtype: int64

Duplicate rows: 0

Dataset after cleaning:
(400, 2)
```

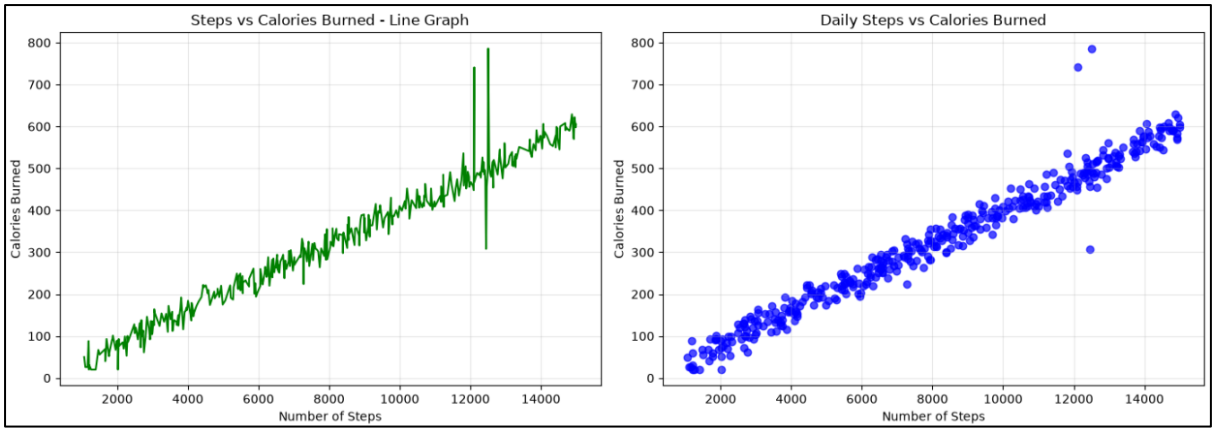
```
sorted_df = df.sort_values("Steps")

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Line graph
axes[0].plot(
    sorted_df["Steps"],
    sorted_df["Calories_Burned"],
    color="green",
    linewidth=1.5
)
axes[0].set_title("Steps vs Calories Burned - Line Graph")
axes[0].set_xlabel("Number of Steps")
axes[0].set_ylabel("Calories Burned")
axes[0].grid(True, alpha=0.3)

# Scatter plot
axes[1].scatter(
    df["Steps"],
    df["Calories_Burned"],
    color="blue",
    alpha=0.7
)
axes[1].set_title("Daily Steps vs Calories Burned")
axes[1].set_xlabel("Number of Steps")
axes[1].set_ylabel("Calories Burned")
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

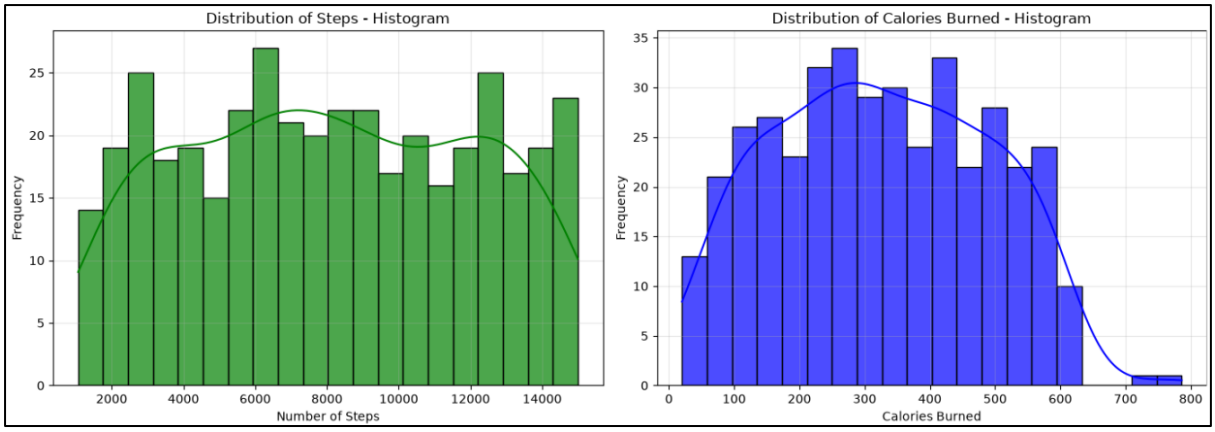



```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Histogram for Steps
sns.histplot(
    data=df,
    x="Steps",
    bins=20,
    color="green",
    edgecolor="black",
    alpha=0.7,
    kde=True,
    ax=axes[0]
)
axes[0].set_title("Distribution of Steps - Histogram")
axes[0].set_xlabel("Number of Steps")
axes[0].set_ylabel("Frequency")
axes[0].grid(True, alpha=0.3)

# Histogram for Calories Burned
sns.histplot(
    data=df,
    x="Calories_Burned",
    bins=20,
    color="blue",
    edgecolor="black",
    alpha=0.7,
    kde=True,
    ax=axes[1]
)
axes[1].set_title("Distribution of Calories Burned - Histogram")
axes[1].set_xlabel("Calories Burned")
axes[1].set_ylabel("Frequency")
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

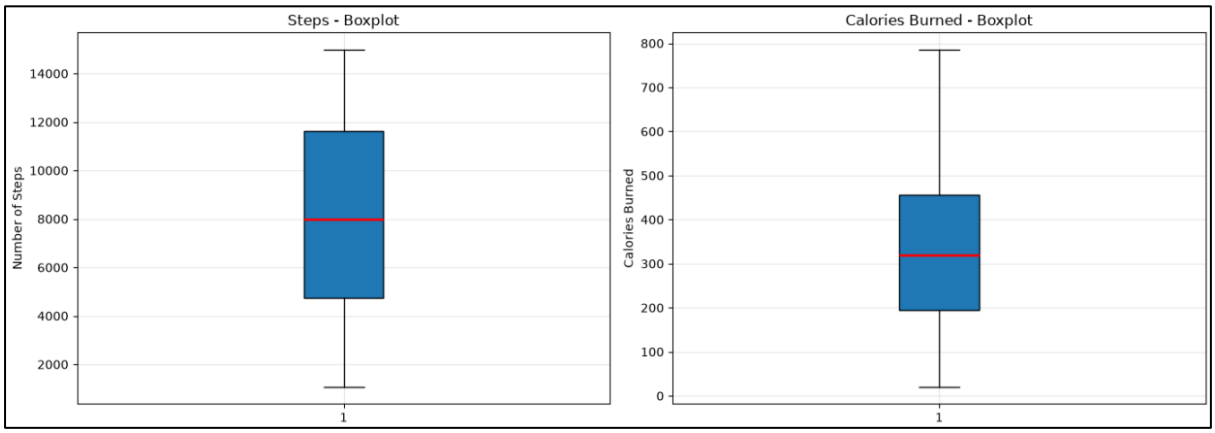



```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Boxplot for Steps
axes[0].boxplot(
    df["Steps"],
    patch_artist=True,
    medianprops=dict(color="red", linewidth=2)
)
axes[0].set_title("Steps - Boxplot")
axes[0].set_ylabel("Number of Steps")
axes[0].grid(True, alpha=0.3)

# Boxplot for Calories Burned
axes[1].boxplot(
    df["Calories_Burned"],
    patch_artist=True,
    medianprops=dict(color="red", linewidth=2)
)
axes[1].set_title("Calories Burned - Boxplot")
axes[1].set_ylabel("Calories Burned")
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



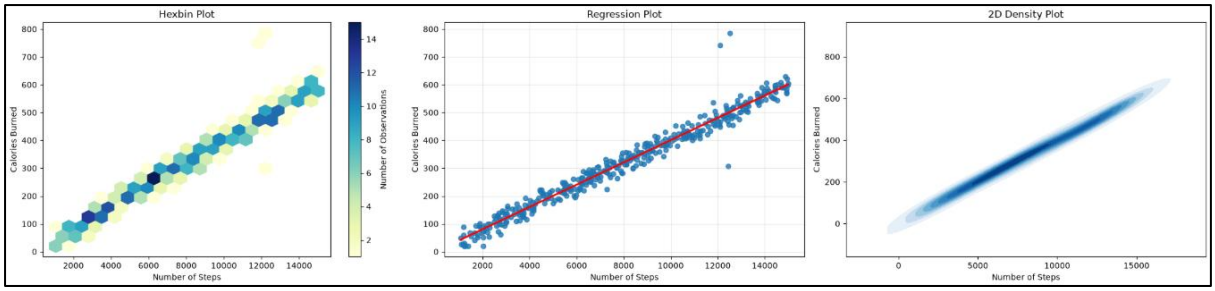
```
fig, axes = plt.subplots(1, 3, figsize=(21, 5))

# 1. Hexbin Plot
hexbin = axes[0].hexbin(
    df["Steps"],
    df["Calories_Burned"],
    gridsize=20,
    cmap="YlGnBu",
    mincnt=1
)
fig.colorbar(hexbin, ax=axes[0], label="Number of Observations")
axes[0].set_title("Hexbin Plot")
axes[0].set_xlabel("Number of Steps")
axes[0].set_ylabel("Calories Burned")

# 2. Regression Plot
sns.regplot(
    data=df,
    x="Steps",
    y="Calories_Burned",
    line_kws={"color": "red"},
    ax=axes[1]
)
axes[1].set_title("Regression Plot")
axes[1].set_xlabel("Number of Steps")
axes[1].set_ylabel("Calories Burned")
axes[1].grid(True, alpha=0.3)

# 3. 2D Density Plot
sns.kdeplot(
    data=df,
    x="Steps",
    y="Calories_Burned",
    fill=True,
    cmap="Blues",
    ax=axes[2]
)
axes[2].set_title("2D Density Plot")
axes[2].set_xlabel("Number of Steps")
axes[2].set_ylabel("Calories Burned")

plt.tight_layout()
plt.show()
```




```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.ecdfplot(data=df, x="Steps", ax=axes[0], color="green")
axes[0].set_title("ECDF of Steps")
axes[0].set_xlabel("Number of Steps")
axes[0].set_ylabel("Cumulative Proportion")
axes[0].grid(True, alpha=0.3)

sns.ecdfplot(data=df, x="Calories_Burned", ax=axes[1], color="blue")
axes[1].set_title("ECDF of Calories Burned")
axes[1].set_xlabel("Calories Burned")
axes[1].set_ylabel("Cumulative Proportion")
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

